Information Security

Threat assessment of embedded CivetWeb

Love Skön & Max Koste

We dedicate this paper to our beloved friends & family:

"*Believe nothing you hear, and only one half that you see.*"

- Edgar Allan Poe

1. Background and Purpose **3**

2. System/Application Decomposition **3**

2.1 Data Flow Diagram 4

2.2 External Dependencies 6

2.3 Entry Points 6

2.4 Exit Points 6

2.5 Assets to Protect 6

2.6 Trust Levels and Boundaries 7

2.7 Users and Roles 7

3. Threat Identification and Ranking **8**

3.1 STRIDE Thread modeling 8

3.1.1 Spoofing 8

3.1.2 Tampering 8

3.1.3 Repudiation 9

3.1.4 Information Disclosure 9

3.1.5 Denial of Service (DoS) 9

3.1.6 Elevation of privilege 9

3.2 STRIDE Threat Analysis Matrix 9

4. Identify Countermeasures/Mitigation **10**

4.1 Secure Development practises 11

4.2 Mitigation Strategies by Threat Category 11

4.2.1 Tampering / Remote code execution, CVE-2020-27304 , CVE-2025-55763: 11

4.2.2 Denial of Service, CVE-2025-9648: 11

4.2.3 Information Disclosure, CVE-2018-12684: 12

5. References **13**

1. Background and Purpose

CivetWeb is a lightweight, open-source embedded web server library primarily written in C, designed for integration into applications or deployment as a standalone web server [1]. It originated as a fork of the Mongoose web server in August 2013 to preserve the permissive MIT license after Mongoose transitioned to a GPL model. Since then, CivetWeb has diverged through independent development, with over 5,700 commits and contributions from more than 234 developers. The project is hosted on GitHub, and remains actively maintained, with the latest commit as recent as November 2025.

The idea of CivetWeb is to provide a simple and embeddable web server that enables developers to add web functionality to their software without heavy dependencies. Its use is especially common in environments where resources are limited, like IoT devices [1, 2]. It can function as a library for existing C/C++ applications (via its API) or as a self-contained executable, requiring no installation. CivetWeb is platform-agnostic, running on Windows, Linux, macOS, UNIX, iOS, Android, and others.
It is primarily request-based, i.e. it does not normally initiate internet connections on its own, being focused on receiving them, and sending responses [1].

As of March 11th 2026 the stable version of CivetWeb is v1.16, with v1.17 under development [1]. Recent updates are focused on small enhancements and primarily bug fixes, suggesting a mature and stable software.
There are no available statistics on how widespread the use of CivetWeb is, for example the documentation makes no mention of amount of downloads. It is however used in several large software projects like NVIDIA DeepStream SDK [3] and CERN ROOT framework [4].

2. System/Application Decomposition

The internal architecture is single-file centric for ease of compilation, with core source in src, headers in include, and examples in examples [1, 2]. Otherwise the structure is divided into various modules with different responsibilities.

Connection Listener

This module uses a master thread to accept connections and a pool of worker threads to process HTTP requests [1, 5]. It is also responsible for sending the response once the processing is finished. The connections can be either TCP or TLS depending on configuration.

HTTP Parser

This module parses raw HTTP data into structured objects, extracting the URI, HTTP headers, query parameters, and other HTTP data [1].

Authenticator

This module can, if enabled, validate HTTP Digest (which is used to validate user identity), and check hashed passwords stored in the .htpasswd file, associated with authorized users [1, 5, 6]. The URI will then be cross-referenced in the configuration files, where protect_uri is used to map users to permitted URIs.

Router

The router analyzes URIs to determine how disparate requests should be handled, and sends them on to the appropriate module. All modules return some response which the router can then send back toward the connection listener.

Static File Server

This module reads files from the configured document root, it handles MIME types (Multipurpose Internet Mail Extensions) as well as file permissions [1, 2, 5].

CGI Executor

This module can launch external processes that execute CGI (Common Gateway Interface) scripts [1, 5].

Host Interface

This module provides API functions (for example mg_start()) which the host uses to interact with CivetWeb when embedded [2].

2.1 Data Flow Diagram

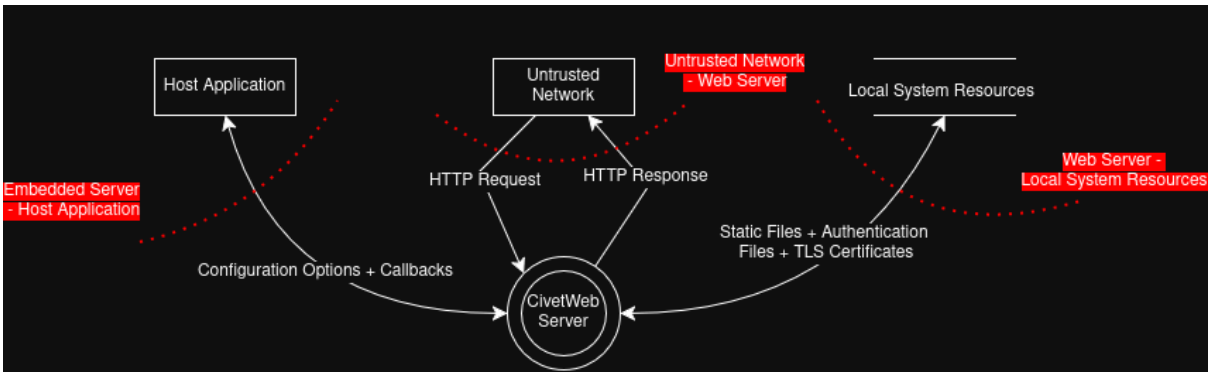

Figure 1: High-Level Architecture Overview

From a low-level perspective, CivetWeb functions as a control center and receiver, picking up HTTP requests, delegating the work required to complete them, before assembling and transmitting a response.

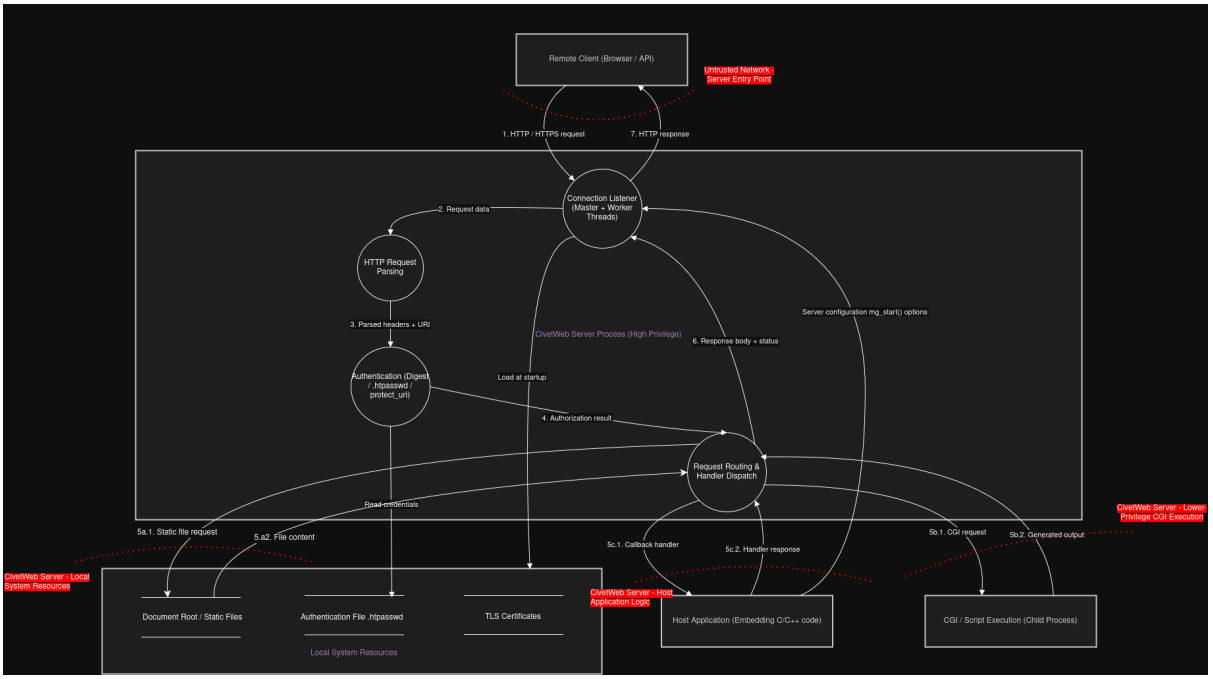

Figure 2: Detailed request lifecycle and privilege boundaries

The above DFD has been marked with a numbered sequence which initiates whenever CivetWeb receives an HTTP request [1]. Configurations and TLS Certificates are loaded at server startup, and are relevant to functionality, but not part of the request flow [6]. The boundaries containing different nodes represent different privilege domains.

1. The initiating step for CivetWeb is receiving an HTTP or HTTPS request from a network.
2. The master thread in the connection listener will receive the job and delegate it to a worker thread which passes it on to the request parser.
3. The request will be broken down into constituent parts before being passed on to the authentication module.
4. The sender's credentials are analyzed here. A user unauthorized for a specific URI would be ejected at this stage, otherwise the system moves on to the router.
5. The router determines how to handle the request the server has been given;
   5a. In the case of a static file request (such as a request for an image), CivetWeb will retrieve it from the local system resources.
   5b. In the case of a CGI request, the contained script will be executed.
   5c. In the case of a custom URI, configured by the host application, CivetWeb will send the request to the host.
6. All routing paths produce some kind of response, which may contain data or simply status, which will then be constructed into an HTTP response ready to be sent back to the network.
7. The same TCP/TLS connection used to receive the request will now send the response, and the process will await new requests.

Depending on the configuration of the CivetWeb server, additional routing paths may be available, but they are not part of the core CivetWeb code and thus are not discussed here [5].

2.2 External Dependencies

There are no required dependencies inside of CivetWeb, as it is intended to be as lightweight as possible. However there are widely used optional dependencies which have significant impacts on security.

In terms of vulnerabilities, additional server side scripting (seperate from the CGI already included) can be implemented with Lua Server Pages. While providing substantial functionality, it of course increases the risk of script injection [1, 5].

For security, it offers HTTPS support through TLS libraries like OpenSSL, Mbed TLS, or GNU TLS, including optional X.509 certificate authentication [1, 6].

2.3 Entry Points

- Network sockets: CivetWeb receives and analyzes HTTP requests.
- Host API calls: The host CivetWeb is embedded into interacts with it through calling its API functions
- File system access: CivetWeb initiates contact with the file system (when loading .htpasswd or reading static files), and then receives data.
- CGI execution: While CGI is being executed, it has trusted access to the application.

2.4 Exit Points

- HTTP responses: Once the processing of a request is finished, a reponse is sent.
- CGI child process execution: CivetWeb launches an external program as a child process.
- File reads: When reading files the system exits CivetWeb's internal structure.
- Host application callbacks: If a request is routed to a custom URI handler, CivetWeb invokes a callback function implemented in the host application.

2.5 Assets to Protect

CivetWeb contains or has access to a variety of potentially valuable assets [1].

A1 - Hashed passwords: Stored in .htpasswd, CivetWeb keeps a list of authorized users and their passwords.
A2 - Files in the document root: CivetWeb has access to the entire document root which can contain a variety of valuable files.
A3 - Application data returned by handlers: CivetWeb may return application-specific data generated by host application handlers, which can include valuable information.
A4 - TLS: When TLS is enabled private encryption keys are stored within CivetWeb, as well

as TLS certificates.
A5 - Server configuration: The servers configuration files, which may contain custom URIs set by the host, and other settings that expose vulnerabilities.
A6 - Host application logic: Through CivetWeb, it is possible to learn more about the host, which may contain valuable assets of its own.
A7 - Worker threads: The threads that process HTTP requests, as well as those that execute CGI scripts.
A8 - Network sockets: CivetWeb maintains open network sockets to communicate with clients.
A9 - System memory: CivetWeb stores request data, configuration information, connection states, and response buffers in system memory during execution.

2.6 Trust Levels and Boundaries

High trust inside of the application boundary and host.

Application: When CivetWeb is embedded in an application it becomes part of that application, so there are no checks in place when moving between CivetWeb and the host (i.e. high trust). It is up to the developer to implement these if they wish to further increase security [1, 5].

Network: When receiving HTTP requests, CivetWeb exposes itself to an untrusted network client. This is the lowest trust zone, and many authentication and validation checks can be enforced here [1, 6].

Local System Resources: CivetWeb interacts with the local filesystem in order to read static files, credential stores, and TLS certificates [1, 2, 6]. These resources exist outside the server process and may contain sensitive information, so access must be carefully controlled.

Process: When executing CGI scripts, they run as separate processes, meaning they are no longer part of the application boundary [1]. This separation introduces a process boundary where data must be passed between CivetWeb and the external program, which may have different privileges.

2.7 Users and Roles

Web-clients: CivetWeb primarily interacts with web clients that are either anonymous or validated through HTTP Digest.

Application Developers: When embedding CivetWeb it is up to the developer how they wish to integrate it into their application [2, 5]. What dependencies they include and how they handle CivetWeb's functionality can entirely change how secure the system is

System Administrators: CivetWeb includes many configuration and setup options [1, 2, 5]. To make proper use of them an administrator is required to edit these files, such as when adding custom URIs or adding new authorized users to the protect_uri list.

3. Threat Identification and Ranking

The primary attack surface of CivetWeb is the HTTP interface exposed through the connection listener. Since the server accepts untrusted input from network clients, all components involved in request processing become potential targets. This includes the HTTP parser, URI router, static file server, CGI execution interface, and host application callbacks. Because CivetWeb operates as an entry point into the host system, vulnerabilities in request parsing or routing may allow attackers to access sensitive files, execute code, or disrupt service availability.

3.1 STRIDE Thread modeling

To identify potential threats to CivetWeb, the STRIDE threat modeling methodology was selected. STRIDE categorizes threats into six types: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege [7].

STRIDE is suitable for this project since CivetWeb functions as an embedded web server that processes external HTTP requests and interacts with a system's resources such as the file system, and application data [1]. Since CivetWeb acts as a boundary between untrusted network clients and trusted internal components, STRIDE provides a structured way to analyze threats to the system [8].

3.1.1 Spoofing

CivetWeb supports HTTP Digest authentication through .htpasswd files. If weak passwords are used or authentication is incorrectly configured, attackers may impersonate legitimate users and gain access to protected URIs defined through the protect_uri configuration option.

3.1.2 Tampering

CivetWeb processes file paths when serving content from the file system, which introduces the risk of directory traversal attacks, demonstrated in the vulnerability CVE-2020-27304. This vulnerability showed that improper validation of file paths may allow attackers to access files outside the intended document root when running on operating systems other than windows [9].

3.1.3 Repudiation

CivetWeb does provide logging capabilities, but if logging is disabled or poorly configured, it may become difficult to trace malicious requests or identify hackers which hinders investigation and accountability.

3.1.4 Information Disclosure

Since CivetWeb is written in C memory management errors or buffer overflows could allow attackers to retrieve sensitive data. This vulnerability is demonstrated in CVE-2025-55763, where a Buffer overflow in the URI parser allows a remote attacker to achieve remote code execution, potentially exposing sensitive information [10].

Information disclosure is also demonstrated by vulnerability CVE-2018-12684, where an out-of-bounds read in the send_ssi_file function allowed attackers to cause information disclosure through a crafted SSI file [11].

3.1.5 Denial of Service (DoS)

Denial of Service attacks attempt to make the server unavailable to legitimate users by exhausting the system resources. The vulnerability CVE-2025-9648 describes how a malformed HTTP request triggers an infinite loop in a library function, exhausting server resources [12].

3.1.6 Elevation of privilege

In some embedded environments, CivetWeb may run with elevated permissions, especially on IoT devices or systems configured to run services as root. If the attacker manages to escalate privileges they could gain access to the host system.

The vulnerability CVE-2025-55763, as discussed earlier in 3.1.4, also allows attackers to achieve remote code execution via HTTP request [10]. This could lead to the attackers elevating their own privileges inside the system and gaining access to the host.

3.2 STRIDE Threat Analysis Matrix

Threat	Stride Category	Assets Affected	Example Vulnerability	Likelihood	Impact	Severity
Directory traversal allowing file access	Tampering	A2, A5	CVE-2020-27304	High	High	Critical
Buffer overflow in URI parser	Elevation of privilege	A9,A6	CVE-2025-55763	High	High	Critical

Malformed HTTP request causing infinite loop	Denial of service	A7,A8	CVE-2025-9648	Medium	Medium	Medium
Out-of-bounds read in SSI handler	information disclosure	A3,A9	CVE-2018-12684	Medium	Medium	Medium
Weak authentication configuration	Spoofing	A1	N/A	Low	Medium	Low
Lack of logging	Repudiation	A5,A3 (indirect)	N/A	Low	Low	Low

Figure 3: STRIDE Matrix

Likelihood is assessed based on exploitability, exposure to remote attackers, and existence of known vulnerabilities. Impact reflects the potential consequences on system assets in terms of confidentiality, integrity, and availability. Severity is derived from the combination of likelihood and impact.

The highest-severity threats are those that allow remote code execution or memory corruption, as they may allow attackers to take control of the server. These threats are particularly critical in embedded environments where CivetWeb may run with elevated privileges or directly control device functionality. Successful exploitation could therefore compromise the entire host system. Medium severity threats mainly affect availability or confidentiality, while lower-severity threats relate to configuration weaknesses such as insufficient authentication or logging.

4. Identify Countermeasures/Mitigation

Since CivetWeb operates as a boundary between external network clients and internal application logic it therefore processes untrusted network input and interacts directly with the local system resources. This boundary role makes it an attractive target for attackers, especially in high privilege contexts such as IoT devices where CivetWeb may be running with root privileges. To address these threats, we recommend the following mitigations.

4.1 Secure Development practises

Since CivetWeb is implemented in C and directly parses untrusted HTTP input, it is susceptible to memory safety vulnerabilities such as buffer overflows, use-after-free errors or out-of-bounds reads that represent a real security risk. Static analysis tools like clang-tidy, cppcheck, flawfinder, Clang Static Analyser can detect memory safety issues, insecure function usage, and potential buffer overflows in the C codebase [13].
In addition, fuzz testing using tools such as AFL++ or LibFuzzer can automatically generate malformed http requests to test CivetWeb's request parser and identify unexpected crashes or memory corruption vulnerabilities [14].

4.2 Mitigation Strategies by Threat Category

4.2.1 Tampering / Remote code execution, CVE-2020-27304 , CVE-2025-55763:

Preventive:
Directory traversal attacks primarily target the Static File Server module, which processes file paths derived from HTTP URIs. CivetWeb deployments should enforce strict validation of requested file paths by enabling strict document_root and url_rewrite_patterns to ensure all resolved paths remain inside the intended directory. It is also advised to only set the cgi_pattern when necessary. Canonical path normalization should also be performed before accessing the file system to prevent traversal sequences such as …/ [15].

Detective:
To detect attacks CivetWeb should enable detailed logging of request paths, parameters and unexpected input patterns. This is done by enabling access_log_file and error_log_file. There should also be alerts for repeated or anomalous access attempts which may indicate exploitation attempts. Security testing of CivetWeb's deployments can also be performed using web security testing tools such as OWASP ZAP. ZAP allows security testers to intercept, manipulate, and replay HTTP requests to identify vulnerabilities such as injection attacks, authentication weaknesses, or improper input validation [16].

Responsive:
We would recommend using fuzz testing with deployments of CivetWeb in order to identify latent vulnerabilities during development before attackers can exploit them.

4.2.2 Denial of Service, CVE-2025-9648:

Preventive:
Because CivetWeb uses a master thread with a pool of worker threads to process requests, malformed or resource-intensive requests can exhaust the worker thread pool. Limiting the number of simultaneous connections and configuring reasonable request timeouts can prevent attackers from exhausting the thread pool [17]. Mitigate this by configuring the request_timeout_ms and max_request_body variables to make it harder for attackers to exhaust the thread pool.

Detective:
Monitor server resource usage, including CPU, memory, and connection counts, via the host application to detect unusual spikes that might indicate a DoS attack [17].

Responsive:
Implement automated restart or failover mechanism to maintain service availability in the event of an overload or crash.

4.2.3 Information Disclosure, CVE-2018-12684:
Preventive:
To prevent unintended exposure of sensitive data CivetWeb should enforce HTTPS with strong TLS ciphers to protect data in transit [18]. It's also relevant here to enforce strict input validation on file names, query parameters, and request payloads to prevent directory traversal and similar attacks [15]. Information disclosure vulnerabilities may also occur in modules that process dynamic content, such as the SSI handler or CGI executor. It is therefore advised to disable SSI (ssi_pattern) unless required and enforce HTTPS (listening_ports with certificate).

Detective:
Maintain audit trails and logging for access to sensitive resources. This is done by enabling full logging. Additionally, monitor for unusual access patterns, such as repeated attempts to access non-public files.

Responsive:
Keep secure backup copies of sensitive files to recover from data corruption.

5. References

[1] CivetWeb Developers, “CivetWeb GitHub Repository,” GitHub, 2026. [Online]. Available: https://github.com/civetweb/civetweb. Accessed: Mar. 10, 2026.

[2] CivetWeb Project, “Embedding CivetWeb,” GitHub, [Online]. Available: https://github.com/civetweb/civetweb/blob/master/docs/Embedding.md. Accessed: Mar. 10, 2026.

[3] NVIDIA, “NVIDIA DeepStream SDK API Reference: civetweb.h Source File,” NVIDIA Docs, 8.0. [Online]. Available: https://docs.nvidia.com/metropolis/deepstream/8.0/sdk-api/civetweb_8h_source.html. Accessed: Mar. 11, 2026.

[4] ROOT, “ROOT: net/http/civetweb/civetweb.h Source File,” ROOT, v6.26. [Online]. Available: https://root.cern/doc/v626/civetweb_8h_source.html. Accessed: Mar. 11, 2026.

[5] CivetWeb Project, “CivetWeb User Manual,” GitHub. [Online]. Available: https://github.com/civetweb/civetweb/blob/master/docs/UserManual.md. Accessed: Mar. 11, 2026.

[6] CivetWeb Project, “OpenSSL,” GitHub. [Online]. Available: https://github.com/civetweb/civetweb/blob/master/docs/OpenSSL.md. Accessed: Mar. 11, 2026.

[7] S. Hernan, S. Lambert, T. Ostwald, and A. Shostack, “Threat Modeling: Uncover Security Design Flaws Using the STRIDE Approach,” *MSDN Magazine*, pp. 68-75, Nov. 2006.

[8] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN, USA: Wiley, 2014.

[9] National Institute of Standards and Technology, “National Vulnerability Database - CVE-2020-27304 Detail,” National Vulnerability Database. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2020-27304. Accessed: Mar. 10, 2026.

[10] National Institute of Standards and Technology, “National Vulnerability Database - CVE-2025-55763 Detail,” National Vulnerability Database. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2025-55763. Accessed: Mar. 10, 2026.

[11] National Institute of Standards and Technology, “National Vulnerability Database - CVE-2018-12684 Detail,” National Vulnerability Database. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2018-12684. Accessed: Mar. 10, 2026.

[12] National Institute of Standards and Technology, “National Vulnerability Database - CVE-2025-9648 Detail,” National Vulnerability Database. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2025-9648. Accessed: Mar. 10, 2026.

[13] analysis-tools-dev, “static-analysis,” GitHub. [Online]. Available: https://github.com/analysis-tools-dev/static-analysis. Accessed: Mar. 12, 2026.

[14] N. Kazankova, “Memory Safety Bugs: An In-Depth Look At Critical Issues,” Code Intelligence Blog. [Online]. Available: https://www.code-intelligence.com/blog/memory_safety_corruption. Accessed: Mar. 11, 2026.

[15] OWASP, “Input Validation Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html. Accessed: Mar. 10, 2026.

[16] ZAP Project, “ZAP,” GitHub, 2026. [Online]. Available: https://github.com/zaproxy. Accessed: Mar. 11, 2026.

[17] OWASP, “Denial of Service Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Denial_of_Service_Cheat_Sheet.html. Accessed: Mar. 10, 2026.

[18] OWASP, “Transport Layer Security Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html. Accessed: Mar. 10, 2026.